Lctr 18: ARM Processor Status Flags

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Definition of the N, Z, C, and V status flags.
- Reading the status flags in ASM.
- N-bit operations with updated N, Z, C and V flags.
- Examples of hand calculations.

Demo project (can be skipped depending on class time):

- Simulating the additions and subtractions with status update using C
  - Using bit-field construct in C to save the status flags.
  - Code for updating the N, Z, C, and V flags.
  - Tricks for variable assignment involving bit-field.
- Verifying the simulated status flags using ASM functions.

18.1 Introduction

18.2 APSR—the application program status register

18.2.1 Operations on unsigned and signed numbers

- With TC expression of signed numbers, hardware for additions and subtractions for signed numbers are the same as those for unsigned numbers.
- **Instructions for additions and subtractions for unsigned operands are also the same as those for signed operands**. In fact, **Cortex-M processors do not even know if we are performing unsigned or signed operations**.
  - The C flag is for unsigned operations; and the V flag is for signed operations.
  - The processor updates both C and V flags simultaneously.
- Additional flags: N (negative) = 1 if the MSB is 1; Z (zero) = 1 if the value is 0.
- **All** flags for different operations:
  - Unsigned numbers: C, N, and Z.
  - Signed numbers: V, N, and Z.

18.2.2 Operations for updating the status flags

There are two ways to update the status flags. The **first** is to use the `S` version of the ASM instructions, which we have seen before. The generic syntax is

```
[op]s Rd, Rn, Op2
```

where `op` is a specific *operation* instruction we use, such as SUB.

The **second** is to use specific test instructions that will be discussed in the next lecture.

Status flags are updated according to:

- The **value** of the number under test (NUT): the N and Z flags.
- The **condition** of the operation under test (OUT): the C and V flags.

18.2.3 Status flags' bit definition and conditions for updating

Here, we list status flags' bit definition in APSR and the conditions for their updating:

- Bit 31: the N (**N**egative) flag. N = 1 if the MSB of the NUT is 1; otherwise, it will be 0.
- Bit 30: the Z (**Z**ero) flag. Z = 1 if the NUT is zero; otherwise, it will be 0.
- Bit 29: the C (**C**arry/Borrow) flag. This has been discussed before.
- Bit 28: the V (O**v**erflow) flag. V = 1 if the signed addition and subtraction has overflow, otherwise, it will be 0.

18.2.4 Accessing APSR in ASM

To move the value of a general purpose register, say, R0, to APSR to set up the values of the NZCVQ flags, we need to use the *move from (general-purpose) register to special (register)* instruction:

```
    msr     apsr_nzcvq, r0
```

To move the value of APSR to a general purpose register, say, R0 again, we need to use the *move from to special (register) to (general-purpose) register* instruction:

```
    mrs     r0, apsr
```

18.3 Demo project

To practice programming and better understand the principle for updating the status flags, we use a demo project, `fm1s_status_flags_in_c`, to **simulate** how the N, Z, C and V flags work.

18.3.1 Bit-fields in C

```
typedef struct {
    uint32_t _reserve   : 28;   // Reserve 28 bits
    uint32_t V          : 1;    // Bit 28, V flag
    uint32_t C          : 1;    // Bit 29, C flag
    uint32_t Z          : 1;    // Bit 30, Z flag
    uint32_t N          : 1;    // Bit 31, N flag
} APSR_t;                       // Bit-field type for APSR
    flags
```

18.4 Operations on `n`-bit numbers with updated status flags

The setting and clearing of the N, Z, C, and V flags are related, based on the ranges of the numbers, which will be revisited below.

18.4.1 Constants defined for `n`-bit numbers

For notational convenience, we use N instead of `n` in mathematic expressions.

To better describe the principle of status flag assignment, let us define a few constants:

- The cardinality of the N-bit number set: $C_N = 2^N$.
- The minimum N-bit unsigned number: $U_{\text{MIN}} = 0$.
- The maximum N-bit unsigned number: $U_{\text{MAX}} = C_N - 1 = 2^N - 1$.
- The minimum N-bit signed number: $S_{\text{MIN}} = -C_N/2 = -2^{N-1}$.
- The maximum N-bit signed number: $S_{\text{MAX}} = C_N/2 - 1 = 2^{N-1} - 1$.

18.4.2 A revisit of number ranges

We have seen before that the **ranges** of unsigned and signed N-bit numbers before. With the above definitions, we summarize them as below:

- The unsigned N-bit number: $[U_{\text{MIN}}, U_{\text{MAX}}] = [0, S_{\text{MAX}}] \cup [S_{\text{MAX}} + 1, U_{\text{MAX}}]$ or $[0, 2^N - 1]$.
- The signed N-bit number: $[S_{\text{MIN}}, S_{\text{MAX}}] = [S_{\text{MIN}}, -1] \cup [0, S_{\text{MAX}}]$ or $[-2^{N-1}, 2^{N-1} - 1]$.

18.4.3 Number conversion between unsigned and signed

- To convert an N-bit unsigned variable to an N-bit signed variable:
  - If the unsigned variable is greater than S_{MAX}, minus its value by C_N.
  - Otherwise, no change.
- To convert an N-bit signed to an N-bit unsigned:
  - If the signed is negative, add its value by C_N.
  - Otherwise, no change.

18.4.4 Rules for updating various flags

18.4.4.1 Setting or clearing of the C flag

Assume we have two unsigned numbers u_1 and u_2. As we have discussed before, the C flag will be set to 1 if:

- There is a carry in unsigned addition $u_1 + u_2$, which happens when $u_1 + u_2 > 2^N - 1$.
- There is no borrow in unsigned subtraction $u_1 - u_2$, which happens when $u_1 - u_2 \geq 0$.

Otherwise, it will be cleared to 0.

18.4.4.2 Setting or clearing of the V flag

Assume we have two signed numbers s_1 and s_2. The V flag will be set to 1 if there is an overflow with the addition or subtraction:

$$\mathrm{V} = \begin{cases} 0, & \text{if } s_1 \pm s_2 \in [-2^{N-1}, 2^N - 1], \\ 1, & \text{otherwise.} \end{cases}$$

18.4.4.3 Setting or clearing of the N and Z flags

The N and Z flags are handle in an easier way:

- The N flag will be set to 1 if the MSB of result is 1; otherwise, it will be 0.
- The Z flag will be set to 1 if the result is zero; otherwise, it will be 0.

18.4.4.4 Correcting the operation result

We need to get the operation result back to the range of the number expression after the operation, as illustrated below in hand-calculations examples.

18.5 Hand calculation examples

Example 1. Consider the following operations of **unsigned** numbers on a 5-bit system: (a) 8 + 14, (b) 18 + 24, (c) 22 - 13, and (d) 9 - 20. (Note that all the numbers given here can be seen as unsigned numbers; we need to do the necessary conversions to signed.) Perform the above (a) to (d) operations using the decimal-based approach to determine:

- The final operation results *in decimal* when seeing the operands as both unsigned and signed.
- The N, Z, C, and V flags.
- The final results *in binary*.

Solution:

For a 5-bit system, here are the constants: $C_N = 32$; $U_{\text{MIN}} = 0$; $U_{\text{MAX}} = 31$; $S_{\text{MIN}} = -16$; $S_{\text{MAX}} = 15$.

(a.1) Unsigned 8 + 14: u_{temp} = _____ => Results:

- Addition result in decimal:
- Carry: [yes/no], so C flag =
- Result = 0: [yes/no], so Z flag =

(a.2) Signed 8 + 14: S_{temp} = _____ => Results:

- Addition result in decimal:
- Overflow: [yes/no] so V flag =
- Result is negative: [yes/no], so N flag =

Addition results for both cases are the same in binary:

(b.1) Unsigned 18 + 24: u_{temp} = _____ => Results:

- Addition result in decimal:
- Carry: [yes/no], so C flag =
- Result = 0: [yes/no], so Z flag =

(b.2) Signed 18 + 24: s_{temp} = _____ => Results:

- Addition result in decimal:
- Overflow: [yes/no] so V flag =
- Result is negative: [yes/no], so N flag =

Addition results for both cases are the same in binary:

(c.1) Unsigned subtraction 22 - 13: s_{temp} _____ => Results:

- Subtraction result in decimal:
- Borrow: [yes/no], so C flag =
- Result = 0: [yes/no], so Z flag =

(c.2) Signed subtraction 22 - 13: s_{temp} = _____ => Results:

- Subtraction result in decimal:
- Overflow: [yes/no], so V flag =
- Result is negative: [yes/no], so N flag =

Subtraction results for both cases are the same in binary:

(d.1) Unsigned subtraction 9-20: s_{temp} = _____ => Results:

- Subtraction result in decimal:
- Borrow: [yes/no], so C flag =
- Result is 0: [yes/no], so Z flag =

(d.2) Signed subtraction 9 - 20: s_{temp} = _____ => Results:

- Subtraction result in decimal:
- Overflow: [yes/no], so V flag =
- Result is negative: [yes/no], so N flag =

Subtraction results for both cases are the same in binary:

> -:-

Example 2. Consider the following operations on **signed** numbers on a 5-bit system: (a) 5 + 11, (b) (-15) + 11, (c) (-3) - 13, and (d) (-15) - 5. (Note that all the numbers given here can be seen as signed numbers; we need to do the necessary conversions to unsigned.) Perform the above (a) to (d) operations using the decimal-based approach to determine:

- The final operation results in decimal when seeing the operands as both unsigned and signed.
- The N, Z, C, and V flags.
- The final results in binary.

Solution:

For a 5-bit system, here are the constants: $C_N = 32$; $U_{\text{MIN}} = 0$; $U_{\text{MAX}} = 31$; $S_{\text{MIN}} = -16$; $S_{\text{MAX}} = 15$.

(a.1) Unsigned 5 + 11: u_{temp} = _____ => Results:

- Addition result in decimal:
- Carry: [yes/no], so C flag =
- Result = 0: [yes/no], so Z flag =

(a.2) Signed 5 + 11: S_{temp} = _____ => Results:

- Addition result in decimal:
- Overflow: [yes/no] so V flag =
- Result is negative: [yes/no], so N flag =

Addition results for both cases are the same in binary:

(b.1) Unsigned (-15) + 11: u_{temp} = _____ => Results:

- Addition result in decimal:
- Carry: [yes/no], so C flag =
- Result = 0: [yes/no], so Z flag =

(b.2) Signed (-15) + 11: s_{temp} = _____ => Results:

- Addition result in decimal:

- Overflow: [yes/no] so V flag =
- Result is negative: [yes/no], so N flag =

Addition results for both cases are the same in binary:

(c.1) Unsigned subtraction (-3) - 13: s_{temp} _____ => Results:

- Subtraction result in decimal:
- Borrow: [yes/no], so C flag =
- Result = 0: [yes/no], so Z flag =

(c.2) Signed subtraction (-3) - 13: s_{temp} = _____ => Results:

- Subtraction result in decimal:
- Overflow: [yes/no], so V flag =
- Result is negative: [yes/no], so N flag =

Subtraction results for both cases are the same in binary:

(d.1) Unsigned subtraction (-15) - 5: s_{temp} = _____ => Results:

- Subtraction result in decimal:
- Borrow: [yes/no], so C flag =
- Result is 0: [yes/no], so Z flag =

(d.2) Signed subtraction (-15) - 5: s_{temp} = _____ => Results:

- Subtraction result in decimal:
- Overflow: [yes/no], so V flag =
- Result is negative: [yes/no], so N flag =

Subtraction results for both cases are the same in binary:

> -:-

18.6 Simulating n-bit operations using C

To better understand the above concepts, we write C code to simulate the n-bit operations.

18.6.1 Constants defined for n-bit numbers

The constants defined for n-bit numbers given above can be expressed in C. For convenience, we can also defined a mask for confining the N-bit number. They are defined in fm1s_stutus_flags_in_c_fns.h:

```
typedef struct {
    int32_t C_N;    // (1 << N), cardinality of N-bit num set
    int32_t C_N1;   // (1 << (N-1)), C_{N-1}
    int32_t U_MIN; // 0, minimum value of unsigned number
    int32_t U_MAX; // (C_N - 1), max value of unsigned N-bit
    int32_t S_MIN; // (-C_N1), min value of signed N-bit num
    int32_t S_MAX; // (C_N1 - 1), max value of signed N-bit
    uint32_t N_MASK;     // (C_N - 1), MASK of N bits
} NBit_Const_t;     // struct for N-bit constants
```

```
static NBit_Const_t NC;

void mp_initialize_n_bit_const(int N) {
    NC.C_N = (1 << N);          // Cardinality of N-bit num set
    NC.C_N1 = (1 << (N-1));     // C_{N-1}
    NC.U_MIN = 0;               // minimum value of unsigned num
    NC.U_MAX = NC.C_N - 1;      // max value of unsigned N-bit
    NC.S_MIN = -NC.C_N1;        // min value of signed N-bit num
    NC.S_MAX = NC.C_N1 - 1;     // max value of signed N-bit num
    NC.N_MASK = NC.C_N - 1;     // MASK of N bits
}
```

18.6.2 Number conversion between unsigned and signed

Here, we consider the non-standard width numbers, such as 5-bit numbers. Note that we have $N < 32$.

The principle of conversion is the same as that for hand calculation.

Let's take a look at the functions used to *type-cast* numbers to unsigned and signed, as shown below, defined in `fm1s_stutus_flags_in_c_cfns.c`.

```
uint32_t mp_n_bit_input_to_unsigned(int x, int N) {
    return x & NC.N_MASK;
}

int32_t mp_n_bit_input_to_signed(int x, int N) {
    x <<= 32 - N;
    x >>= 32 - N;
    return x;
}
```

- For the first function above, consider an example of converting -2 expressed in a 4-bit system, which is 0b1110, to unsigned. When we load the 4-bit number to R0 according to EBAI, its value is still -2, which is 0xFFFF_FFFE. The value of `NC.N_MASK` is 0b1111, so the result should be 0xE = 0b1110, which is expected.
- For the second function above, consider a 4-bit system with an unsigned number `x` = 0b1110 (=14). When we convert 14 to signed, it is -2. Now, let's see what the function does by calling `mp_n_bit_input_to_signed(x, 4)`. Within the function, we have `32 - N = 28`. So, with `x <<= 28`, we have `x = 0xE000_0000`, and with `x >>= 28`, we have `x = 0xFFFF_FFFE`, which is -1. This verifies the working of the program.

18.6.3 `n+1`-bit result for `n`-bit operations

We can show that to express the addition and subtraction results of N-bit numbers without the risk of overflowing, we need to use $N + 1$-bit system.

18.6.4 Basic unsigned or signed additions and subtractions without status flags

See the Collection.

18.6.5 Simulation of the addition operations in C with status flags

Simulation of the addition operations in C is given in the following function, defined in `fm1s_stutus_flags_in_c_cfns.c`.

```c
uint64_t mp_n_bit_x_plus_y_with_apsr(int32_t x, int32_t y, int N) {
    uint32_t r_u;       // result in unsigned form
    int32_t r_s;        // result in signed form
    APSR_t apsr;        // apsr for status flags

    uint32_t u_temp = mp_n_bit_unsigned_addition(x, y, N);
    if (u_temp > NC.U_MAX) {
        r_u = u_temp - NC.C_N;
        apsr.C = 1;
    } else {
        r_u = u_temp;
        apsr.C = 0;
    }

    int32_t s_temp = mp_n_bit_signed_addition(x, y, N);
    if (s_temp < NC.S_MIN) {
        r_s = s_temp + NC.C_N;
        apsr.V = 1;
    } else if (s_temp > NC.S_MAX) {
        r_s = s_temp - NC.C_N;
        apsr.V = 1;
    } else {
        r_s = s_temp;
        apsr.V = 0;
    }

    apsr.N = r_s >> (N-1); // See demo proj for other method
    apsr.Z = (r_s == 0)? 1 : 0;

    // Assign a bit-field var to a uint32_t var via union
    APSR_32_t apsr_32;
    apsr_32.apsr = apsr;
```

```
    uint64_t result = apsr_32.apsr_val;
    return (result << 32) + r_u;
}
```

Note that `APSR_32_t` is defined below, used to get the value from the bit-field variable.

Similarly to the above addition function, we can write a function to do the subtraction with updated status flags. See the demo project for the implementation.

18.6.6 Programming tricks in using the bit-field data

Bit-fields are used in the above functions to assign values of the status flags. This saves us from the direct bitwise operations. Yet, to assign the bit-field variable value to an common variable, we have to use some tricks. *The first*, shown in the above code, uses an union, defined below.

```
typedef union {
    APSR_t apsr;         // APSR flags
    uint32_t apsr_val;   // 32-bit value
} APSR_32_t;             // Union for APSR and its 32-bit value
```

The second, shown in the corresponding subtraction function, uses a pointer, as illustrated below.

```
    // Assign the value of a bit-field var to a uint32_t var
       via pointer
    uint64_t result = *(uint32_t *)&apsr;
    return (result << 32) + r_u;
```

Note that the last line of code combines two 32-bit variables into a single 64-bit one. This is a trick we can use when we need to return two values from a single ASM function.

We can see that the pointer-based approach is more compact.

18.7 Verifying n-bit operations using ASM

To ensure we have the correct values for the status flags, we can use the ASM functions to verify them. These functions are provided in `fm1s_stutus_flags_in_c_sfns.s`.

Below is the ASM version of `mp_n_bit_x_plus_y_with_apsr` we discussed previously:

```
@ 32-bit signed addition with status flags updated
@ uint64_t mp_n_bit_adds_s(int32_t x, int32_t y, int N)
@   Input:
@     * x -> r0, y -> r1, N -> r2
@   Output:
@     * r1:r0
@       * r0: x + y
@       * r1: APSR
mp_n_bit_adds_s:
    rsb     r2, r2, #32  @ R2 = 32 - N
    lsl     r0, r2       @ R0 = R0 << R2
    lsl     r1, r2       @ R1 = R1 << R2
    adds    r0, r1       @ R0 = R0 + R1
    asr     r0, r2       @ R0 = R0 >> R2
    mrs     r1, apsr     @ R1 = APSR
    bx      lr
```

The major difference between the C and ASM functions for addition is that the later does not need to worry about the status flags, which are handled by the processor automatically.